



Universidade do Minho
Escola de Engenharia

Relatório do Trabalho Prático

Bases de Dados Clínicas e de Gestão Hospitalar

Licenciatura em Engenharia Biomédica

Grupo 10:

A101472, Alexandra Mendes

A101470, Filipe Tavares

A101481, Lara Santos

Docentes:

José Machado

Hugo Peixoto



Índice

1	Introdução	1
1.1	Contextualização	1
1.2	Apresentação do Caso de Estudo	1
1.3	Motivação e Objetivos	2
2	Desenvolvimento da Base de Dados	2
2.1	Desenho de uma Base de Dados	2
2.1.1	Diagrama Conceitual	2
2.1.2	Modelo Lógico	4
2.1.3	Modelo Físico	5
2.2	Povoamento	6
2.3	Queries	7
2.4	Subprogramas SQL	10
2.4.1	Funções	11
2.4.2	Triggers	12
2.4.3	Procedimentos	17
3	Conclusão	20
	Referências	20
A	Anexos	22
A.1	Tabela de Atributos	22
A.2	Modelo físico	22
A.3	Povoamento	25
A.4	<i>Queries</i>	26



1 Introdução

1.1 Contextualização

A manutenção regular de equipamentos médicos em hospitais é fundamental para assegurar a segurança dos pacientes e a eficiência dos serviços de saúde. Equipamentos em condições subótimas podem levar a diagnósticos incorretos, tratamentos inadequados e, em casos extremos, colocar vidas em risco. Dada a diversidade e a complexidade dos aparelhos utilizados em ambientes hospitalares, é imprescindível dispor de um sistema organizado que facilite o agendamento e o acompanhamento das manutenções preventivas e corretivas.

Um sistema bem estruturado para a gestão dessas manutenções permite um controlo rigoroso sobre o estado dos equipamentos, antecipando falhas, minimizando interrupções nos serviços e garantindo que todos os aparelhos estejam sempre em condições ideais de funcionamento. Além disso, a rastreabilidade detalhada dos históricos de manutenção permite identificar tendências de desgaste e pontos críticos que necessitam de atenção especial, promovendo intervenções mais precisas e eficazes.

A utilização de uma plataforma centralizada para monitorizar e gerenciar a manutenção dos equipamentos médicos também oferece benefícios adicionais, dado que é possível a monitorização dos custos de cada intervenção de manutenção e os gastos adjacentes a esta. Estes incluem a otimização do uso de recursos e a redução de custos associados a reparos emergenciais ou substituições prematuras. Em última análise, assegurar a manutenção adequada dos equipamentos não só melhora a segurança e a qualidade do atendimento prestado, mas também contribui para a sustentabilidade operacional e financeira do hospital, promovendo um ambiente de cuidado mais seguro e confiável para todos os pacientes.

1.2 Apresentação do Caso de Estudo

O caso de estudo selecionado é usado como forma representativa daquilo que seria um Sistema de Gestão de Base de Dados de um hospital. Este está focado maioritariamente na manutenção de dispositivos médicos, bem como nas suas intervenções de manutenção e custos associados. O sistema referido é usado para monitorizar as intervenções feitas aos equipamentos médicos do hospital em questão, contendo entidades como, a ordem de serviço de uma intervenção, a própria intervenção, bem como as peças usadas nesta e o técnico responsável por ela, o histórico de intervenção de cada equipamento, o equipamento referido, o departamento a que pertence e o responsável por este.



1.3 Motivação e Objetivos

Este trabalho tem como principal objetivo consolidar a aprendizagem dos conteúdos abordados no planeamento e execução de projetos de Sistemas de Bases de Dados relacionais. Para isso, vamos analisar um problema específico, planear e modelar uma solução através dos modelos conceitual, lógico e físico, e realizar a sua implementação prática em várias etapas.

Durante a implementação, faremos o povoamento do modelo e desenvolveremos três tipos de subprogramas em PL/SQL (*Procedural Language/Structured Query Language*): Procedimentos, Funções e *Triggers*. Esses subprogramas serão utilizados para realizar consultas e atualizações de informações que foram consideradas pertinentes no contexto do caso de estudo escolhido. Dessa forma, também contribuirão para alcançar os objetivos estabelecidos.

Para atingir o objetivo proposto e aplicá-lo a um caso prático de grande relevância no mundo atual, o grupo prosseguiu com o desenvolvimento deste trabalho. Além disso, o caso de estudo apresenta relações lógicas bem definidas, justificando sua implementação em modelos relacionais de maneira relativamente simples.

2 Desenvolvimento da Base de Dados

2.1 Desenho de uma Base de Dados

No desenvolvimento de uma base de dados, é fundamental estruturar um raciocínio lógico que orientará a sua criação, visando minimizar erros ao longo do processo.

Para a projeção de uma base de dados, este processo divide-se em 3 fases essenciais:

- Diagrama Conceitual;
- Modelo Lógico;
- Modelo Físico.

2.1.1 Diagrama Conceitual

Numa fase inicial é essencial identificar as entidades a serem analisadas, juntamente com os seus atributos, e estabelecer as relações entre elas, de forma a refletir a lógica subjacente ao pensamento humano.

Na notação Chen, as entidades são representadas por retângulos verdes, os atributos por elipses roxas e as relações entre entidades por losangos amarelos contidos em retângulos verdes. Já os atributos-chave são sublinhados e em negrito para destaque.

De maneira a definir as relações, a cardinalidade é crucial indicando o número de ocorrências de uma relação. As relações entre entidades podem ser de três tipos: 1 para 1 (1:1),



1 para muitos (1:N) e muitos para muitos (N:N). Este último tipo é utilizado quando a mesma entidade pode participar em várias populações-alvo e, ao mesmo tempo, uma mesma população-alvo pode ser mediada por diferentes entidades, gerando uma entidade-relação.

Em relação às linhas que ligam as entidades, caso estas sejam simples, indicam que a relação não é obrigatória, caso contrário, a obrigatoriedade é representado por uma linha dupla.

Com isto em mente, a base de dados criada, representada na Figura 1, é composta por 11 entidades, sendo que 2 destas são entidades relacionais.

Desta maneira, sendo a base de dados direcionada a monitorizar a manutenção de equipamentos, foi criada a entidade de *Intervenção de Manutenção* que tem como atributos associados o seu id, a descrição do que foi realizado nesta intervenção, o custo da mesma, bem como o custo total, ou seja, um atributo derivado que soma o custo da intervenção com os gastos das peças utilizadas nestas. Para além destes, tem ainda a data de início e a de fim, de forma a dar a conhecer a duração desta intervenção.

Associada à intervenção, está o técnico que a realizou, sendo *Tecnico_de_manutencao* outra entidade. A este estão associados como atributos o seu nome, data de nascimento e o seu contacto, que permite a sua contratação.

Além do técnico, está também associado à intervenção as peças que foram utilizadas nesta. Assim, foi criada a entidade *Peca* com os atributos designação e preço, que será usado na tabela *Intervencao_de_Manutencao* para calcular o custo total da intervenção. A estas peças está associado um fornecedor das mesmas, que gerou a necessidade de criar a entidade *Fonecedor_de_peca* juntamente com a sua entidade relacional *Fornecedor_Peca*. Esta primeira tem como atributos o nome e o contacto do fornecedor, enquanto que a segunda tem como atributo a data de compra da peça em si.

Para a intervenção de manutenção ser realizada é necessário que haja uma ordem de serviço por parte do hospital, o que gerou a entidade *Ordem_de_servico*, que engloba a prioridade com que a intervenção tem de ser realizada, a descrição desta ordem, o estado atual do desenvolvimento da ação e a data de criação da mesma.

Esta ordem contém também qual o equipamento que necessita de intervenção, gerando-se outra entidade *Equipamento*, que possui como atributos a sua designação, a data de aquisição do mesmo e a data de quando foi a sua última manutenção, permitindo ao hospital decidir se o equipamento necessita de ser substituído conforme a regularidade das suas manutenções. O que torna imprescindível a tabela de *Historico_de_Manutencao* à qual está associada a tabela de *Equipamento* e de *Intervencao_de_Manutencao*, e tem como atributo o custo, permitindo ao hospital consultar o dinheiro gasto em intervenções.

Em relação ao equipamento, este pertence a um departamento (entidade *Departamento*), que possui uma localização no hospital e uma designação. Como entidade relação entre estas

duas, foi criada a entidade *Departamento_Equipamento* cujo atributo é a data de aquisição do equipamento pelo departamento.

Por fim, foi também criada a entidade *Responsavel* referente ao responsável do departamento, que possui nome e data de nascimento.

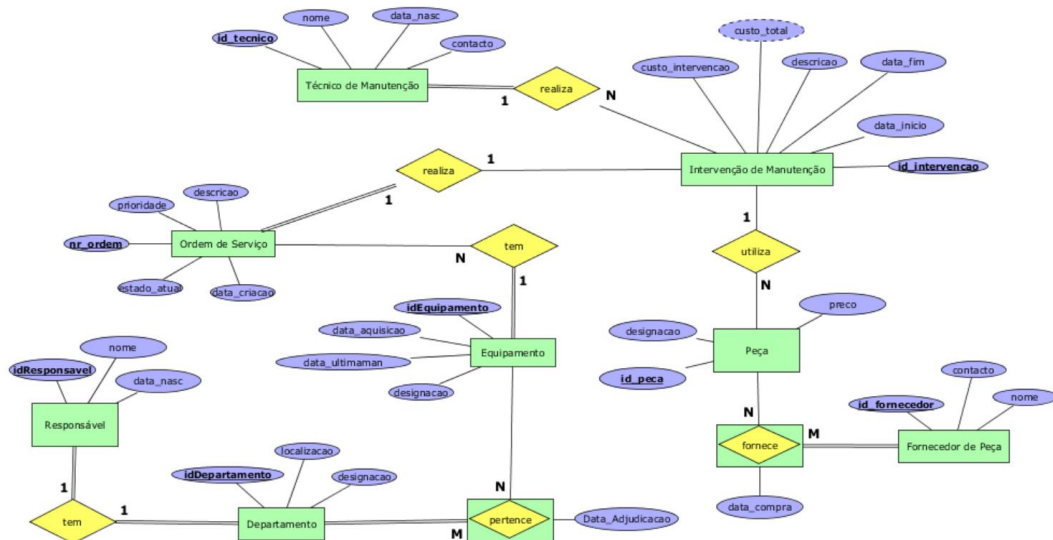


Figura 1: Desenho Conceitual da base de dados.

2.1.2 Modelo Lógico

O modelo lógico de uma base de dados consiste em converter o modelo conceptual num modelo de dados representacional, utilizando um Sistema de Gestão de Base de Dados (SGBD). Este modelo é uma representação simplificada e intuitiva da base de dados que se pretende implementar, como é possível verificar na Figura 2. Através de processos de normalização, obtém-se um conjunto de tabelas simples que permitem um registo correto dos dados.

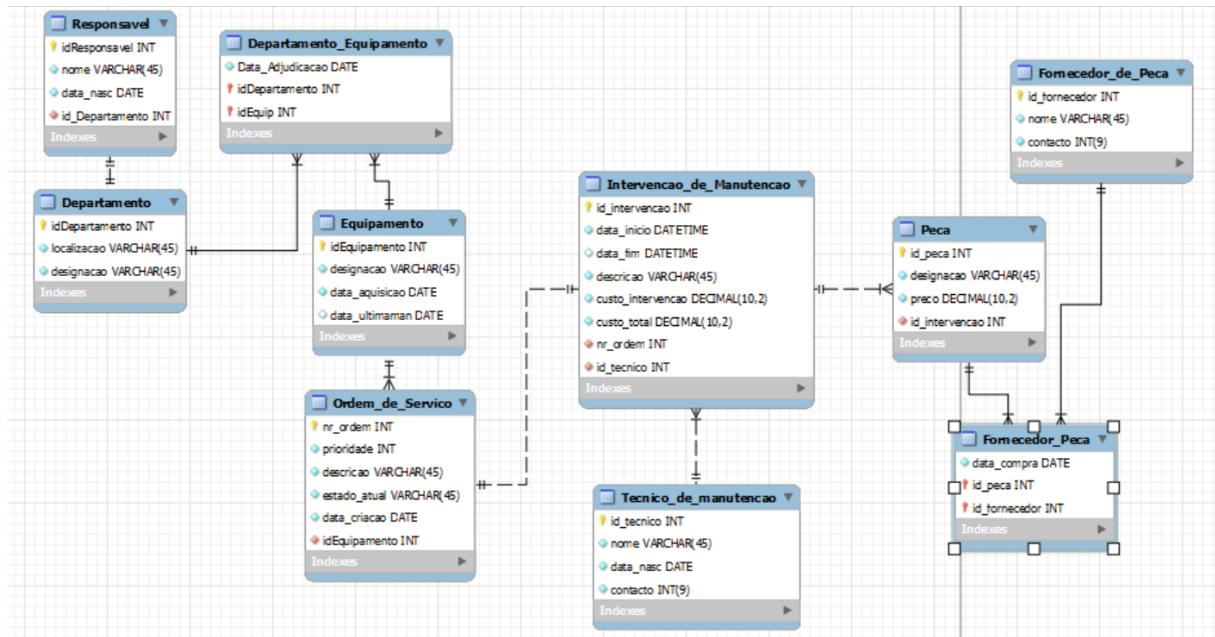


Figura 2: Modelo Lógico da base de dados.

2.1.3 Modelo Físico

O modelo físico de uma base de dados possibilita a implementação do modelo lógico, previamente construído, no armazenamento secundário. Este modelo, Figura 3, foi criado utilizando uma funcionalidade do *SQLWorkbench*, a exportação por meio do *Forward Engineer SQL Script*. Ainda nesta figura, podemos analisar a criação de uma tabela no modelo físico. A criação das restantes tabelas encontra-se no Anexo A2.

```

-- MySQL Workbench Forward Engineering
SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';

-- Schema mydb

CREATE SCHEMA IF NOT EXISTS `mydb` DEFAULT CHARACTER SET utf8 ;
USE `mydb` ;

-- Table `mydb`.`Equipamento`

CREATE TABLE IF NOT EXISTS `mydb`.`Equipamento` (
  `idEquipamento` INT NOT NULL,
  `designacao` VARCHAR(45) NOT NULL,
  `data_aquisicao` DATE NOT NULL,
  `data_ultimaman` DATE NULL,
  PRIMARY KEY (`idEquipamento`))
ENGINE = InnoDB;
  
```

Figura 3: *Script* do modelo físico da base de dados e da criação da tabela *Equipamento*.

Inicialmente, define-se a condição para a criação da tabela, caso ela ainda não exista (*CREATE TABLE IF NOT EXISTS*), especificando os atributos, bem como o tipo destes e se aceitam valores nulos ou não. As chaves primárias são definidas pelo comando *PRIMARY*



KEY seguido de parênteses que contêm o atributo ou os atributos que serão as chaves primárias.

De maneira a inserir chaves estrangeiras são usados os comandos *FOREIGN KEY* e *REFERENCES*, como se pode ver na Figura 4. Neste caso a coluna *idEquipamento* em *Ordem_de_Servico* é definida como uma chave estrangeira que referencia a coluna *idEquipamento* na tabela *Equipamento*.

```
-- Table 'mydb`.`Ordem_de_Servico`  
-----  
CREATE TABLE IF NOT EXISTS `mydb`.`Ordem_de_Servico` (  
  `nr_ordem` INT NOT NULL,  
  `prioridade` INT NOT NULL,  
  `descricao` VARCHAR(45) NOT NULL,  
  `estado_atual` VARCHAR(45) NOT NULL,  
  `data_criacao` DATE NOT NULL,  
  `idEquipamento` INT NOT NULL,  
  PRIMARY KEY (`nr_ordem`),  
  INDEX `idEquipamento_idx` (`idEquipamento` ASC) VISIBLE,  
  CONSTRAINT `idEquipamento`  
    FOREIGN KEY (`idEquipamento`)  
      REFERENCES `mydb`.`Equipamento` (`idEquipamento`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION  
  ENGINE = InnoDB;
```

Figura 4: *Script* do modelo físico da base de dados que contém chaves estrangeiras.

Com a criação de todas as tabelas e a execução do *script*, é possível visualizar o desenho físico pretendido na Figura 5.

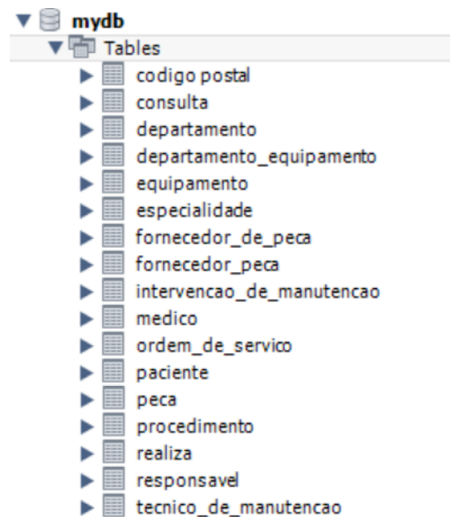


Figura 5: Desenho Físico da base de dados.

2.2 Povoamento

Após a criação das tabelas, é necessário povoar as mesmas, isto é, inserir dados nas tabelas da base de dados, preenchendo-as com registros que contêm as informações relevantes.

A instrução *SQL* usada para povoar uma tabela é o comando *INSERT INTO*. Na Figura

6, é possível ver a inserção de dados nas tabelas *Equipamento* e *Departamento*. Na tabela *Equipamento*, são adicionados cinco equipamentos com seus respectivos identificadores, designações, datas de aquisição e datas da última manutenção, enquanto que na tabela *Departamento*, são inseridos cinco departamentos, cada um com seu identificador, localização e designação. O povoamento das restantes tabelas encontram-se no Anexo A3.

```
-- Populando a tabela Equipamento
INSERT INTO Equipamento (idEquipamento, designacao, data_aquisicao, data_ultimaman) VALUES
(1, 'Raio-X', '2019-01-15', '2022-05-21'),
(2, 'Impressora', '2022-02-20', '2023-01-22'),
(3, 'Monitor Cardíaco', '2020-03-10', '2024-01-03'),
(4, 'Mesa de cirurgia', '2022-04-05', '2023-06-07'),
(5, 'Bomba de Infusão', '2021-05-12', '2023-09-01');

-- Populando a tabela Departamento
INSERT INTO Departamento (idDepartamento, localizacao, designacao) VALUES
(1, 'Ala A', 'Ortopedia'),
(2, 'Ala B', 'Clínica Geral'),
(3, 'Ala C', 'Cardiologia'),
(4, 'Ala D', 'Pediatria'),
(5, 'Ala E', 'Neurologia');
```

Figura 6: Povoamento.

2.3 Queries

Uma *query* é definida como uma instrução utilizada para interagir com uma base de dados. Estas permitem realizar uma variedade de operações, como filtrar dados com base em condições específicas, combinar informações de diferentes tabelas através de *joins*, calcular estatísticas ou ainda de ordenar resultados, entre outras funcionalidades [1].

Fornecedor de Id 3

De maneira a se identificar qual o fornecedor de peças que possui o identificador número 3, é utilizado o *query* da Figura 7.

Este seleciona o nome e o contacto do fornecedor (fo.nome, fo.contacto) relacionado à peça específica utilizada na intervenção, identificada pelo seu *ID* (pe.id_intervencao). Para isso, a *query* requer a tabela *Fornecedor_de_Peca*, *Fornecedor_Peca* e *Peca*, usando condições de junção (*JOIN*) para combinar os registos relevantes. Através do comando *WHERE*, os *ID*'s do fornecedor e da peça são associados à intervenção desejada, permitindo a recuperação dos dados do fornecedor correspondente.

Na Figura 8, está representada a resposta a esta solicitação.



```
-- 1 Fornecedor da intervencao de id 3  
SELECT fo.nome, fo.contacto FROM fornecedor_de_peca fo, fornecedor_peca fop, peca pe  
WHERE fo.id_fornecedor= fop.id_fornecedor AND fop.id_peca = pe.id_peca AND  
pe.id_intervencao = 3;
```

Figura 7: *Query* de Identificação do Fornecedor de ID 3

	nome	contacto
►	Fornecedor C	934567890

Figura 8: Resultado do *Query* de Identificação do Fornecedor de ID 3

Seleção das Ordens Existentes e o seu Estado

Esta *query* tem como objetivo recuperar informações sobre as ordens de serviço existentes, incluindo o número da ordem (os.nr_ordem), a prioridade (os.prioridade), a descrição (os.descricao) e o estado atual (os.estado_atual).

Representada na Figura 9, ela realiza uma junção (*JOIN*) entre a tabela *Ordem_de_Servico* e a tabela *Equipamento*, onde os registos são combinados com base no *ID* do equipamento associado a cada ordem de serviço. Essa junção permite correlacionar as ordens de serviço com os equipamentos correspondentes, possibilitando a recuperação das informações sobre o estado das ordens. A listagem destas estão na Figura 10.

```
-- 2 As ordens existentes e o seu estado  
SELECT os.nr_ordem, os.prioridade, os.descricao, os.estado_atual  
FROM Ordem_de_Servico os  
JOIN Equipamento e ON os.idEquipamento = e.idEquipamento;
```

Figura 9: *Query* de Seleção das Ordens Existentes e o seu Estado.

	nr_ordem	prioridade	descricao	estado_atual
►	1	1	Manutenção Preventiva	Em Andamento
	2	2	Substituição de Peças	Pendente
	3	3	Calibração	Concluída
	4	1	Emergência	Em Andamento
	5	2	Reparo	Concluída

Figura 10: Resultado do *Query* de Seleção das Ordens Existentes e o seu Estado.

Seleção dos Equipamentos por Departamento

De forma a recuperar informações sobre os equipamentos associados a cada departamento, foi criada a *query* da Figura 11.

Esta seleciona o *ID* do equipamento (e.idEquipamento), a sua designação (e.designacao), a designação do departamento (d.designacao), e a data de adjudicação (de.Data_Adjudicacao) do equipamento. A junção é realizada entre as tabelas *Equipamento* e *Departamento_Equipamento*, usando a correspondência dos *ID*'s dos equipamentos. De seguida, é feita outra junção com a tabela *Departamento*, onde os registos são combinados com base nos *ID*'s dos departamentos associados aos equipamentos. Os resultados desta consulta são visíveis na Figura 12.



```
-- 3 Equipamentos por departamento
SELECT e.idEquipamento, e.designacao, d.designacao AS departamento, de.Data_Adjudicacao
FROM Equipamento e
JOIN Departamento_Equipamento de ON e.idEquipamento = de.idEquip
JOIN Departamento d ON de.idDepartamento = d.idDepartamento;
```

	idEquipamento	designacao	departamento	Data_Adjudicacao
▶	1	Raio-X	Ortopedia	2019-01-15
	2	Impressora	Clínica Geral	2022-02-20
	3	Monitor Cardíaco	Cardiologia	2020-03-10
	4	Mesa de cirurgia	Pediatria	2022-04-05
	5	Bomba de Infusão	Neurologia	2021-05-12

Figura 11: *Query* de Seleção dos Equipamentos por Departamento.

Figura 12: Resultado do *Query* de Seleção dos Equipamentos por Departamento.

Seleção das Intervenções que estão a decorrer

A *query*, representada na Figura 13, é projetada para recuperar informações sobre as intervenções de manutenção que estão atualmente "Em Andamento". Ela seleciona o *ID* da intervenção (*im.id_intervencao*), a data de início (*im.data_inicio*), a descrição (*im.descricao*), o custo da intervenção (*im.custo_intervencao*), o custo total (*im.custo_total*) e o nome do técnico responsável (*t.nome*). A junção é feita entre as tabelas *Intervencao_de_Manutencao* e *Tecnico_de_manutencao*, utilizando a correspondência dos *ID*'s dos técnicos. O comando *WHERE* filtra os registros onde a data de fim da intervenção (*im.data_fim*) é nula, indicando que a intervenção ainda não foi concluída. O resultado desta consulta está na Figura 14.

```
-- 4 Intervenções a decorrer
SELECT im.id_intervencao, im.data_inicio, im.descricao, im.custo_intervencao, im.custo_total, t.nome AS tecnico
FROM Intervencao_de_Manutencao im
JOIN Tecnico_de_manutencao t ON im.id_tecnico = t.id_tecnico
WHERE im.data_fim IS NULL;
```

Figura 13: *Query* da Seleção das Intervenções que estão a decorrer.

	id_intervencao	data_inicio	descricao	custo_intervencao	custo_total	tecnico
▶	1	2024-01-15 08:00:00	Substituição de Tubo	200.00	300.00	João Silva
	4	2024-04-20 11:00:00	Substituição de Motor	400.00	550.00	Ana Santos

Figura 14: Resposta da *Query* da Seleção das Intervenções que estão a decorrer.

Seleção das Peças usadas nas Intervenções

De forma a listar informações sobre as peças utilizadas em intervenções de manutenção, juntamente com os detalhes dos fornecedores dessas peças, a *query*, representada na Figura 15, seleciona o *ID* da intervenção (*im.id_intervencao*), a descrição da intervenção (*im.descricao*), a designação da peça (*p.designacao*), o preço da peça (*p.preco*), o nome do fornecedor (*f.nome*), e o contacto do fornecedor (*f.contacto*). A junção é realizada entre as tabelas *Intervencao_de_Manutencao*, *Peca*, *Fornecedor_Peca* e *Fornecedor_de_Peca*, onde os registros são combinados com base nos *ID*'s das intervenções de manutenção, peças e



fornecedores associados. O comando *ORDER BY* ordena os resultados com base na data de início da intervenção (*im.data_inicio*) em ordem decrescente, possibilitando a visualização das informações mais recentes primeiro. Por fim, os resultados desta pesquisa encontram-se na Figura 16.

```
-- 11 Peças usadas nas intervenções
SELECT
    im.id_intervencao,
    im.descricao AS Descricao_Intervencao,
    p.designacao AS Peca,
    p.preco AS Preco_Peca,
    f.nome AS Fornecedor,
    f.contacto AS Contacto_Fornecedor
FROM
    Intervencao_de_Mantencao im
JOIN
    Peca p ON im.id_intervencao = p.id_intervencao
JOIN
    Fornecedor_Peca fp ON p.id_pecas = fp.id_pecas
JOIN
    Fornecedor_de_Peca f ON fp.id_fornecedor = f.id_fornecedor
ORDER BY
    im.data_inicio DESC;
```

Figura 15: *Query* de Seleção das Peças usadas nas Intervenções.

	id_intervencao	Descricao_Intervencao	Peca	Preco_Peca	Fornecedor	Contacto_Fornecedor
►	5	Reparo de Circuito	Placa de Circuito	150.00	Fornecedor E	956789012
	4	Substituição de Motor	Motor de Rotação	150.00	Fornecedor D	945678901
	3	Calibração do Equipamento	Sensor de Calibração	200.00	Fornecedor C	934567890
	1	Substituição de Tubo	Tubo de Raio-X	100.00	Fornecedor A	912345678

Figura 16: Resultado do *Query* de Seleção das Peças usadas nas Intervenções.

As restantes *queries* que, por exemplo, abordam o custo de todas as intervenções num intervalo de tempo ou as intervenções cujo custo se encontra acima da média de preço, encontram-se no Anexo A4.

2.4 Subprogramas SQL

PL/SQL é uma extensão da linguagem padrão *SQL* para o *SGBD Oracle* da *Oracle Corporation*. É uma linguagem procedural da *Oracle* que estende a linguagem *SQL*. Permite que a manipulação de dados seja incluída em unidades de programas.

Em *PL/SQL* existem três tipos de subprogramas fundamentais:

- Funções;
- Procedimentos;
- Triggers.



2.4.1 Funções

Funções são um dos tipos de subprogramas mencionados anteriormente, capazes de manipular dados numa perspetiva de consulta, tornando a informação obtida mais útil e eficiente [2].

As funções *SQL* são implementadas com *SQL* e podem ser usadas para encapsular a lógica que pode ser invocada como uma sub-rotina de programação. Estas possuem diversas aplicações úteis dentro de uma base de dados ou na arquitetura de aplicações da base de dados, como a criação de operadoras em dados de coluna, a ampliação de suporte de funções integradas, além de melhorar o *design* e a segurança da base de dados.

Cálculo da Idade

A função *calcular_idade*, representada na Figura 17, é criada para calcular a idade com base na data de nascimento fornecida (*data_nasc*). Ela retorna um valor inteiro que representa a diferença em anos entre a data de nascimento e a data atual (*CURDATE()*), usando a função *TIMESTAMPDIFF*. Esta função é utilizada em consultas *SQL* de maneira a determinar a idade de indivíduos diretamente a partir de suas datas de nascimento.

```
DELIMITER $$
CREATE FUNCTION calcular_idade(data_nasc DATE)
RETURNS INT
BEGIN
    RETURN TIMESTAMPDIFF(YEAR, data_nasc, CURDATE());
END$$
DELIMITER ;
```

Figura 17: Função de Cálculo da Idade.

Cálculo do Custo Total da Manutenção

A função *custo_total Equipamento*, representada na Figura 18, calcula o custo total de manutenção para um equipamento específico, identificado pelo seu *idEquipamento*. Ela retorna um valor decimal com precisão de duas casas decimais. A função soma os custos totais de todas as intervenções de manutenção (*custo_total*) associadas ao equipamento, utilizando uma junção entre as tabelas *Intervencao_de_Manutencao* e *Ordem_de_Servico*. Se não houver intervenções de manutenção para o equipamento especificado, a função retorna 0, utilizando *IFNULL* para tratar valores nulos.

É possível verificar o seu funcionamento na Figura 19.

```
DELIMITER $$
CREATE FUNCTION custo_total Equipamento(idEquipamento INT)
RETURNS DECIMAL(10,2)
BEGIN
    DECLARE total DECIMAL(10,2);
    SELECT SUM(im.custo_total) INTO total FROM Intervencao_de_Manutencao im
    JOIN Ordem_de_Servico os ON im.nr_ordem = os.nr_ordem
    WHERE os.idEquipamento = idEquipamento;
    RETURN IFNULL(total, 0);
END$$
DELIMITER ;
```

Figura 18: Função de Cálculo do Custo Total da Manutenção.

	custo_total Equipamento(1)
▶	1250.00

Figura 19: Custo total do equipamento de ID 1

Cálculo do tempo médio de todas as intervenções

A função *calcular_tempo_medio_intervencao*, representada na Figura 20, oferece uma métrica valiosa para avaliar o desempenho das operações de manutenção. Ao fornecer uma compreensão clara do tempo médio necessário para concluir uma intervenção, as equipas de manutenção podem identificar áreas de melhoria, otimizar processos e garantir uma alocação eficiente de recursos. É possível verificar o seu funcionamento na Figura 21.

```
DELIMITER //
CREATE FUNCTION calcular_tempo_medio_intervencao()
RETURNS DECIMAL(10, 2)
BEGIN
    DECLARE tempo_medio DECIMAL(10, 2);

    SELECT AVG(TIMESTAMPDIFF(MINUTE, data_inicio, data_fim)) INTO tempo_medio
    FROM Intervencao_de_Manutencao
    WHERE data_fim IS NOT NULL;

    RETURN IFNULL(tempo_medio, 0.00);
END //
DELIMITER ;
```

Figura 20: Função que calcula o tempo médio das intervenções

	calcular_tempo_medio_intervencao()
	405.00

Figura 21: Tempo médio de todas as intervenções em minutos

2.4.2 Triggers

Os *triggers* em base de dados são mecanismos avançados que executam automaticamente ações predefinidas em resposta a operações *SQL* como *UPDATE*, *INSERT* ou *DELETE*,

numa tabela. Este tipo de procedimento é essencial para aplicar regras de negócios, validar dados de entrada, e manter trilhas de auditoria. Podem ser configurados antes, *before triggers*, ou depois, *after triggers*, da operação que os aciona, permitindo um controlo detalhado sobre o comportamento da base de dados. A sua implementação requer uma compreensão aprofundada das operações de dados e da lógica de negócios para assegurar eficiência e evitar sobrecargas desnecessárias [3].

Validação da existência de Ordem de Serviço antes da inserção de uma Intervenção

Para verificar a validação da inserção de uma nova intervenção é necessário que uma ordem de serviço referente a esta exista previamente. Para tal criou-se o *trigger* demonstrado na Figura 22.

```
DELIMITER //
CREATE TRIGGER before_insert_intervencao
BEFORE INSERT ON Intervencao_de_Mantencao
FOR EACH ROW
BEGIN
    DECLARE ordem_existente INT;

    SELECT COUNT(*)
    INTO ordem_existente
    FROM Ordem_de_Servico
    WHERE nr_ordem = NEW.nr_ordem;

    IF ordem_existente = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Não é possível adicionar uma intervenção de manutenção sem uma ordem de serviço correspondente.';
    END IF;
END;
//
DELIMITER ;
```

Figura 22: *Trigger* de Validação da existência de Ordem de Serviço antes da inserção de uma Intervenção.

Este *trigger* é ativado antes da inserção de um novo registo na tabela da intervenção, verificando a existência de uma ordem de serviço com *nr_ordem* correspondente ao do novo registo. Caso contrário, gera um erro com a mensagem "Não é possível adicionar uma intervenção de manutenção sem uma ordem de serviço correspondente". Este *trigger* assegura que não seja possível adicionar uma intervenção de manutenção sem uma ordem de serviço válida. É possível verificar o seu funcionamento nas figuras 23 e 24.

```
INSERT INTO Intervencao_de_Mantencao (id_intervencao, data_inicio, data_fim, descricao, custo_intervencao, custo_total, nr_ordem, id_tecnico) VALUES
(11, '2024-06-02 09:00:00', NULL, 'Substituição de Tubo', 200.00, 300.00, 50, 3);
```

Figura 23: Tentativa de inserção de uma intervenção relativa a uma ordem inexistente.

```
INSERT INTO Intervencao_de_Mantencao (id_intervencao, data_inicio, data_fim, descricao, custo_intervencao, custo_total, nr_ordem, id_tecnico) VALUES
(11, '2024-06-02 09:00:00', NULL, 'Substituição de Tubo', 200.00, 300.00, 50, 3); Error Code: 1644. Não é possível adicionar uma intervenção de manutenção sem uma ordem de serviço corre...
```

Figura 24: Erro na inserção da intervenção.

Atualização do estado_atual na Ordem_Serviço

De modo a efetuar a atualização do estado de uma intervenção, foi necessário a criação de diversos *triggers* capazes de fazer esta mudança na tabela *Ordem_de_Servico* de forma automática, no caso de alterações em atributos da tabela *Intervencao_de_Manutencao*.

Assim, estabeleceram se os *triggers* expostos nas Figuras 25, 26 e 27.

```
DELIMITER //
CREATE TRIGGER after_insert_intervencao
AFTER INSERT ON Intervencao_de_Manutencao
FOR EACH ROW
BEGIN
    -- Verifica se data_fim não é NULL
    IF NEW.data_fim IS NOT NULL THEN
        -- Verifica se data_fim é maior que data_inicio
        IF NEW.data_fim > NEW.data_inicio THEN
            -- Atualiza o estado da ordem de serviço para 'Concluída'
            UPDATE Ordem_de_Servico
            SET estado_atual = 'Concluída'
            WHERE nr_ordem = NEW.nr_ordem;
        ELSE
            -- Gera um erro se data_fim não for maior que data_inicio
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Data de fim tem de ser superior à de inicio';
        END IF;
    END IF;
END //
DELIMITER ;
```

Figura 25: *Trigger* de inserção do estado da ordem de serviço.

```
DELIMITER //
CREATE TRIGGER after_update_intervencao
AFTER UPDATE ON Intervencao_de_Manutencao
FOR EACH ROW
BEGIN
    IF NEW.data_fim IS NOT NULL AND OLD.data_fim IS NULL THEN
        UPDATE Ordem_de_Servico
        SET estado_atual = 'Concluída'
        WHERE nr_ordem = NEW.nr_ordem;
    END IF;
END;
//
DELIMITER ;
```

Figura 26: *Trigger* de atualização do estado da ordem de serviço para 'Concluída'.

```
DELIMITER //
CREATE TRIGGER update_ordem_estado AFTER INSERT ON Intervencao_de_Manutencao
FOR EACH ROW
BEGIN
    IF NEW.data_fim IS NULL THEN
        UPDATE Ordem_de_Servico
        SET estado_atual = 'Em Andamento'
        WHERE nr_ordem = NEW.nr_ordem;
    END IF;
END;
//
DELIMITER ;
```

Figura 27: *Trigger* de atualização do estado da ordem de serviço para 'Em Andamento'.

O primeiro *trigger*, é ativado automaticamente após a inserção de um novo registo na tabela *Intervencao_de_Manutencao*. Este verifica se a *data_fim* não é nula e se é posterior à *data_inicio*. Caso se verifique ambas as condições, o estado da *Ordem_Servico* é atualizada para "Concluída", caso contrário, gera um erro. É possível verificar o seu funcionamento nas Figuras 28, 29 e 30.

nr_ordem	prioridade	descricao	estado_atual	data_criacao	idEquipamento
10	1	Manutenção Preventiva	Pendente	2024-05-29	1

Figura 28: Estado atual pendente antes da inserção da intervenção.

```
INSERT INTO Intervencao_de_Manutencao (id_intervencao, data_inicio, data_fim, descricao, custo_intervencao, custo_total, nr_ordem, id_tecnico) VALUES
(10, '2024-05-30 08:00:00', '2024-05-30 10:00:00', 'Substituição de Tubo', 200.00, 300.00, 10, 1)
```

Figura 29: Inserção da intervenção, com data inicio e data fim definidas.



nr_ordem	prioridade	descricao	estado_atual	data_criacao	idEquipamento
10	1	Manutenção Preventiva	Concluída	2024-05-29	1

Figura 30: Estado atual concluído após a inserção da intervenção.

Já o segundo *trigger*, é ativado após uma atualização na tabela referida anteriormente, verificando se a nova *data_fim* não é nula, e se a antiga *data_fim* o é. Se estas condições forem satisfeitas, atualiza o estado da *Ordem_de_Serviço* para "Concluída". Este *trigger* assegura que o estado de ordem de serviço é atualizado adequadamente quando a intervenção é concluída. É possível verificar o seu funcionamento nas Figuras 31, 32 e 33.

nr_ordem	prioridade	descricao	estado_atual	data_criacao	idEquipamento
1	1	Manutenção Preventiva	Em Andamento	2024-01-10	1

Figura 31: Estado atual "Em Andamento" antes da atualização da data de fim.

```
Update intervencao_de_manutencao  
Set data_fim = '2024-01-15 09:00:00'  
Where nr_ordem = 1;
```

Figura 32: Atualização da *data_fim*.

nr_ordem	prioridade	descricao	estado_atual	data_criacao	idEquipamento
1	1	Manutenção Preventiva	Concluída	2024-01-10	1

Figura 33: Estado atual concluído após a atualização da data de fim

O *trigger* seguinte verifica que a *data_fim* de um novo registo é nula. Se for nula, o estado da *Ordem_de_Serviço* correspondente atualiza para "Em Andamento". Este *trigger* assegura que as ordens de serviço são marcadas como "Em Andamento" quando a intervenção de manutenção associada ainda não foi finalizada. É possível verificar o seu funcionamento nas Figuras 34, 35 e 36.

nr_ordem	prioridade	descricao	estado_atual	data_criacao	idEquipamento
2	2	Substituição de Peças	Pendente	2024-05-29	4

Figura 34: Estado atual pendente antes de ser adicionada uma intervenção.

```
INSERT INTO Intervencao_de_Manutencao (id_intervencao, data_inicio, data_fim, descricao, custo_intervencao, custo_total, nr_ordem, id_tecnico) VALUES  
(15, '2024-06-02 09:00:00', NULL, 'Substituição de Tubo', 200.00, 300.00, 2, 3);
```

Figura 35: Inserção da intervenção, com data inicio e data fim definidas.



nr_ordem	prioridade	descricao	estado_atual	data_criacao	idEquipamento
2	2	Substituição de Peças	Em Andamento	2024-05-29	4

Figura 36: Estado atual "Em Andamento" após a inserção da intervenção.

Atualização da Data da Última Manutenção de Equipamento

De maneira a que a data da última manutenção no equipamento seja atualizada na tabela *Intervencao_de_Manutencao* quando uma intervenção é realizada, foi criado o *trigger* apresentado na Figura 37.

```
DELIMITER $$
CREATE TRIGGER ultima_int
BEFORE INSERT ON Intervencao_de_Manutencao
FOR EACH ROW
BEGIN
    DECLARE v_idEquipamento INT;
    -- Obtém o idEquipamento associado à ordem de serviço
    SELECT idEquipamento INTO v_idEquipamento FROM Ordem_de_Servico WHERE nr_ordem = NEW.nr_ordem;
    -- Atualiza a data da última manutenção no equipamento
    UPDATE Equipamento
    SET data_ultimaman = NEW.data_inicio
    WHERE idEquipamento = v_idEquipamento;
END$$
DELIMITER ;
```

Figura 37: *Trigger* de Atualização da Data da Última Manutenção de Equipamento.

Este *trigger* é acionado antes de uma inserção na tabela *Intervencao_de_Manutencao*. Por cada nova linha inserida, ele obtém o *idEquipamento* associado à ordem de serviço (*nr_ordem*) da nova intervenção. De seguida, atualiza a coluna *data_ultimaman* na tabela *Equipamento* com a data de início (*data_inicio*) da nova intervenção, garantindo que a data da última manutenção do equipamento esteja sempre atualizada. É possível verificar o seu funcionamento nas Figuras 38, 39 e 40.

idEquipamento	designacao	data_aquisicao	data_ultimaman
1	Raio-X	2019-01-15	2024-01-15

Figura 38: Data da última manutenção antes da inserção da intervenção.

```
INSERT INTO Ordem_de_Servico (nr_ordem, prioridade, descricao, estado_atual, data_criacao, idEquipamento) VALUES
(20, 1, 'Manutenção Preventiva', 'Concluído', '2024-04-29', 1);
INSERT INTO Intervencao_de_Manutencao (id_intervencao, data_inicio, data_fim, descricao, custo_intervencao, custo_total, nr_ordem, id_tecnico) VALUES
(20, '2024-05-02 09:00:00', '2024-05-02 10:00:00', 'Substituição de Tubo', 200.00, 300.00, 20, 3);
```

Figura 39: Inserção da intervenção, com data início e data fim definidas.

idEquipamento	designacao	data_aquisicao	data_ultimaman
1	Raio-X	2019-01-15	2024-05-02

Figura 40: Data da última manutenção após a inserção da intervenção.

Verificação da existência de uma Intervenção associada a uma Ordem de Serviço

Para que não haja mais que uma ordem de serviço para uma intervenção, foi criado o *trigger* da Figura 41, que previne essa duplicação.

```
DELIMITER //
CREATE TRIGGER verificar_ordem_intervencao
BEFORE INSERT ON Intervencao_de_Manutencao
FOR EACH ROW
BEGIN
    DECLARE count_existing INT;
    -- Verifica se o nr_ordem fornecido já está associado a outra intervenção
    SELECT COUNT(*) INTO count_existing
    FROM Intervencao_de_Manutencao
    WHERE nr_ordem = NEW.nr_ordem;
    -- Se já existir uma intervenção associada a esse nr_ordem, gera um erro
    IF count_existing > 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Já existe uma intervenção associada a este número de ordem.';
    END IF;
END //
DELIMITER ;
```

Figura 41: *Trigger* de Verificação da existência de uma intervenção associada a uma ordem de serviço.

O *trigger*, *verificar_ordem_intervencao*, é acionado antes de uma inserção na tabela *Intervencao_de_Manutencao*. Por cada nova linha inserida, ele verifica se o número da ordem de serviço (*nr_ordem*) fornecido já está associado a outra intervenção na tabela. Se encontrar alguma intervenção existente com o mesmo número de ordem, o *trigger* gera um erro com a mensagem "Já existe uma intervenção associada a este número de ordem", impedindo a duplicação de ordens de serviço. É possível verificar o seu funcionamento nas Figuras 42, 43.

```
INSERT INTO Intervencao_de_Manutencao (id_intervencao, data_inicio, data_fim, descricao, custo_intervencao, custo_total, nr_ordem, id_tecnico) VALUES
(25, '2024-05-02 09:00:00', '2024-05-02 10:00:00', 'Substituição de Tubo', 200.00, 300.00, 1, 3);
```

Figura 42: Tentativa de inserção de uma intervenção, com número de ordem já existente.

```
INSERT INTO Intervencao_de_Manutencao (id_intervencao, data_inicio, data_fim, descricao, custo_intervencao, custo_total, nr_ordem, id_tecnico) VALUES
(25, '2024-05-02 09:00:00', '2024-05-02 10:00:00', 'Substituição de Tubo', 200.00, 300.00, 1, 3); Error Code: 1644. Já existe uma intervenção associada a este número de ordem.
```

Figura 43: Erro na inserção devido à já existência de uma ordem.

2.4.3 Procedimentos

Os procedimentos fazem parte do grupo de subprogramas do *PL/SQL* e são linhas de código *SQL* que podem ser guardadas e usadas sempre que forem requeridas.

Estes procedimentos são compostos por instruções cuja ordem de definição importa, permitindo ao programador verificar condições para atualizar ou observar os dados que cumprem essas condições [4].

Os procedimentos podem aceitar parâmetros de entrada, retornar parâmetros de saída ou até parâmetros que sejam simultaneamente dos dois tipos.

Existem muitas aplicações úteis para procedimentos SQL dentro de um banco de dados ou na arquitetura de aplicativos de banco de dados, como a criação de *scripts* para consulta rápida, transformação e atualização de dados, geração de relatórios básicos, melhoria do desempenho do aplicativo, modularização de aplicações, e melhoria do *design* e da segurança do banco de dados.

Procedimento para Atualização do Contacto do Técnico

O procedimento da Figura 44, atualiza o número do contacto telefónico de um técnico de manutenção. Este recebe como parâmetros o identificador do técnico, *p_id_tecnico*, e o novo número de contacto, *p_novo_contacto*, atualizando o campo *contacto* na tabela *Tecnico_de_Manutencao* para o técnico correspondente.

```
DELIMITER //
CREATE PROCEDURE atualizar_contacto_tecnico(
    IN p_id_tecnico INT,
    IN p_novo_contacto INT(9)
)
BEGIN
    -- Update the technician's contact number
    UPDATE Tecnico_de_manutencao
    SET contacto = p_novo_contacto
    WHERE id_tecnico = p_id_tecnico;
END //
DELIMITER ;
```

Figura 44: Procedimento de atualização do contacto de um técnico.

Na Figura 45 encontram-se os dados referentes à tabela de *Tecnico_de_Manutencao* antes da realização do procedimento.

	id_tecnico	nome	data_nasc	contacto
▶	1	João Silva	1990-05-15	912345678

Figura 45: Contacto atual do técnico de manutenção.

Para recorrer a um procedimento é necessário chamá-lo, para o exemplo fornecido usou-se o comando da Figura 46.

```
call atualizar_contacto_tecnico(1, 987654321);
```

Figura 46: Sintaxe para chamar o procedimento *atualizar_contacto_tecnico*.

Após a realização do comando pode-se verificar que o contacto do técnico de ID=1 sofreu a alteração desejada (Figura 47).

	id_tecnico	nome	data_nasc	contacto
▶	1	João Silva	1990-05-15	987654321

Figura 47: Contacto atualizado do técnico de manutenção após o procedimento.

Procedimento para Criação de um Responsável de Departamento

No procedimento da Figura 48, um novo responsável para um departamento é criado. Este procedimento recebe parâmetros como o nome, p_nome , a data de nascimento do novo responsável, p_data_nasc , e o identificador do departamento, $p_id_departamento$. Primeiramente será verificado a existência de um departamento. Em caso negativo, um erro será gerado. No caso do departamento existir, o responsável anterior será eliminado, e o novo responsável obtém o próximo *ID* disponível, inserindo o novo responsável na tabela *Responsavel*.

```

DELIMITER //
CREATE PROCEDURE criar_responsavel_departamento(
    IN p_nome VARCHAR(45),
    IN p_data_nasc DATE,
    IN p_id_departamento INT
)
BEGIN
    DECLARE Id INT;

    -- Verify if the department exists
    IF (SELECT COUNT(*) FROM Departamento WHERE idDepartamento = p_id_departamento) = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Department not found';
    END IF;

    -- Delete the previous responsible for the department
    DELETE FROM Responsavel WHERE id_Departamento = p_id_departamento;

    -- Get the next available id for the new responsible
    SELECT MAX(idResponsavel) INTO Id FROM Responsavel;
    SET Id = Id + 1;

    -- Insert the new responsible for the department
    INSERT INTO Responsavel (idResponsavel, nome, data_nasc, id_Departamento)
    VALUES (Id, p_nome, p_data_nasc, p_id_departamento);
END //
DELIMITER ;

```

Figura 48: Procedimento para criar um responsável para um departamento.

Na Figura 49 encontram-se os dados referentes à tabela de *Responsavel* antes da realização do procedimento em questão.

	idResponsavel	nome	data_nasc	id_Departamento
▶	1	Carlos Silva	1980-03-15	1
	2	Maria Santos	1975-06-20	2
	3	Lara Mangualde	1980-07-21	3
	4	Cristiano Ronaldo	1970-09-22	4
	5	Alexandra Freitas	1987-10-10	5
•	NULL	NULL	NULL	NULL

Figura 49: Dados relativos à tabela *Responsavel* antes do comando do procedimento.

De forma a testar o procedimento, este teve de ser chamado, com recurso à sintaxe da Figura 50.

```
CALL criar_responsavel_departamento("Carla Mendes","2003-01-01",1);
```

Figura 50: Sintaxe do procedimento *criar_responsavel_departamento*.

Uma vez realizado o comando, o procedimento foi capaz de adicionar o responsável desejado, removendo o anterior do departamento de ID=1, como demonstrado na Figura 51.

	idResponsavel	nome	data_nasc	id_Departamento
▶	2	Maria Santos	1975-06-20	2
	3	Lara Mangualde	1980-07-21	3
	4	Cristiano Ronaldo	1970-09-22	4
	5	Alexandra Freitas	1987-10-10	5
	6	Carla Mendes	2003-01-01	1
*	NULL	NULL	NULL	NULL

Figura 51: Resultado do procedimento *criar_responsavel_departamento*.

3 Conclusão

O trabalho permitiu consolidar os conceitos de gestão de bases de dados aprendidos previamente. Através da análise de um problema específico, planeamos e modelamos uma solução utilizando os modelos conceitual, lógico e físico, seguindo-se a implementação prática em várias etapas. Durante a implementação, realizamos o povoamento do modelo e desenvolvemos três tipos de subprogramas em *PL/SQL*: Procedimentos, Funções e *Triggers*. Esses subprogramas foram fundamentais para realizar consultas e atualizações pertinentes no contexto do caso de estudo escolhido, contribuindo significativamente para o alcance dos objetivos estabelecidos.

A base de dados escolhida mostrou boa capacidade de atualização, sendo dinâmica e respondendo adequadamente às atualizações requeridas. Adicionalmente, o sistema permitiu a gestão eficiente de informações relacionadas à manutenção de equipamentos médicos, facilitando o acesso e a manipulação dos dados.

Outros pontos que poderíamos melhorar incluem a criação de mais *triggers*, especialmente para a validação de dados inseridos, bem como *triggers* que enviem alertas ou notificações em caso de falhas nos equipamentos. Estas melhorias potencializariam ainda mais a eficácia e a segurança do sistema de gestão hospitalar.

Em suma, este projeto facilitou a aplicação e a melhor compreensão dos tópicos estudados, proporcionando uma experiência prática valiosa no desenvolvimento e gestão de bases de dados clínicas e hospitalares.



Referências

- [1] Andrei L. Query em Bancos de Dados: Guia Rápido e Prático. <https://www.hostinger.com.br/tutoriais/o-que-e-query>, 2023. [Accessed 28-05-2024].
- [2] Funções SQL. <https://www.ibm.com/docs/pt-br/db2/11.1?topic=sr-sql-functions>, 2023. [Accessed 25-05-2024].
- [3] Tipos de triggers. <https://www.ibm.com/docs/pt-br/db2/11.1?topic=triggers-types>, 2023. [Accessed 28-05-2024].
- [4] Procedimentos SQL. <https://www.ibm.com/docs/pt-br/db2/11.1?topic=sr-sql-procedures>, 2023. [Accessed 28-05-2024].



A Anexos

A.1 Tabela de Atributos

Entidade	Atributos	Descrição Atributos	Tipos de Dados	Nulo	Multi-valor	Chave Primária	Chave estrangeira	Atributo composto
Equipamento	idEquipamento	Identificador equipamento	Inteiro	Não	Não	Sim	Não	Não
	designação	Designação equipamento	45 caracteres	Não		Não		
	data_aquisicao	Data aquisição do equip.	Data	Não		Não		
	data_ultimaman	Data última manutenção	Data	Sim		Não		
Departamento	idDepartamento	Identificador departamento	Inteiro	Não	Não	Sim	Não	Não
	localização	Localização do departamento	45 caracteres			Não		
Ordem_de_Servico	designacao	Nome do departamento	45 caracteres	Não	Não	Não	Não	Não
	nr_ordem,	Número da ordem,	Inteiro			Sim		
	prioridade,	Prioridade da ordem,	Inteiro			Não		
	descricao,	Descrição do serviço	45 caracteres			Não		
	estado_atual,	Estado de conclusão	45 caracteres			Não		
	data_criacao,	Quando foi criada	Data			Não		
Tecnico_de_Manutencao	idEquipamento	Número do equipamento	Inteiro	Não	Não	Não	Sim	Não
	Id_tecnico	Identificador do técnico	Inteiro			Sim		
	Nome	Nome do técnico	45 caracteres			Não		
	Data_nasc	Data nascimento	Data			Não		
Intervencao_de_Manutencao	contacto	Contacto do técnico	9 algarismos	Não	Não	Não	Não	Não
	Id_Intervencao	Identificador da intervenção	Inteiro			Sim		
	data_inicio	Data início intervenção	Data e hora			Não		
	data_fim	Data fim intervenção	Data e hora			Sim		
	descricao	Custo levado pelo técnico	45 caracteres			Não		
	custo_intervencao	Custo total da intervenção	10 algarismos, 2 casas decimais			Não		
	custo_total	Número da ordem de serviço	10 algarismos, 2 casas decimais			Não		
	nr_ordem	Identificador do técnico	Inteiro			Não		
	id_tecnico		Inteiro			Não		
						Sim		
Peca	Id_peca	Identificador da peça	Inteiro	Não	Não	Sim	Não	Não
	Designação	Designação da peça	45 caracteres			Não		
	Preco	Custo da peça	10 algarismos, 2 casas decimais			Não		
	Id_intervencao	Identificador da intervenção	Inteiro			Sim		
Fornecedor_de_Peca	Id_fornecedor	Identificador do fornecedor	Inteiro	Não	Não	Sim	Não	Não
	Nome	Nome do fornecedor	45 caracteres			Não		
Fornecedor_Peca	contacto	Contacto do fornecedor	9 algarismos	Não	Não	Não	Sim	Não
	Data_compra	Data de compra da peça	Data			Sim		
Departamento_Equipamento	Id_peca	Identificador da peça	Inteiro	Não	Não	Não	Sim	Não
	Id_fornecedor	Identificador do fornecedor	Inteiro			Sim		
	Data_Adjudicacao	Data aquisição do equipamento	Data			Sim		
Responsavel	idDepartamento	Identificador do departamento	Inteiro	Não	Não	Não	Sim	Não
	idEquip	Identificador do equipamento	Inteiro			Não		
	idResponsavel	Identificador do responsável	Inteiro			Sim		
	nome	Nome do responsável	45 caracteres			Não		
	data_nasc	Data nascimento	Data	Não	Não	Não	Sim	Não
	id_Deartamento	Identificador departamento	Inteiro			Não		

Figura 52: Tabela de Atributos

A.2 Modelo físico

```
CREATE TABLE IF NOT EXISTS `mydb`.`Responsavel` (  
  `idResponsavel` INT NOT NULL,  
  `nome` VARCHAR(45) NOT NULL,  
  `data_nasc` DATE NOT NULL,  
  `id_Departamento` INT NOT NULL,  
  PRIMARY KEY (`idResponsavel`),  
  INDEX `idDepartamento_idx` (`id_Departamento` ASC) VISIBLE,  
  CONSTRAINT `id_Departamento`  
    FOREIGN KEY (`id_Departamento`)  
    REFERENCES `mydb`.`Departamento` (`idDepartamento`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 53: Criação da tabela *Responsavel*.



```
CREATE TABLE IF NOT EXISTS `mydb`.`Fornecedor_Peca` (  
  `data_compra` DATE NOT NULL,  
  `id_pecas` INT NOT NULL,  
  `id_fornecedor` INT NOT NULL,  
  PRIMARY KEY (`id_pecas`, `id_fornecedor`),  
  INDEX `id_fornecedor_idx` (`id_fornecedor` ASC) VISIBLE,  
  CONSTRAINT `id_pecas`  
    FOREIGN KEY (`id_pecas`)  
      REFERENCES `mydb`.`Peca` (`id_pecas`)  
      ON DELETE NO ACTION  
      ON UPDATE NO ACTION,  
  CONSTRAINT `id_fornecedor`  
    FOREIGN KEY (`id_fornecedor`)  
      REFERENCES `mydb`.`Fornecedor_de_Peca` (`id_fornecedor`)  
      ON DELETE NO ACTION  
      ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 54: Criação da tabela *Fornecedor_Peca*.

```
CREATE TABLE IF NOT EXISTS `mydb`.`Departamento_Equipamento` (  
  `Data_Adjudicacao` DATE NOT NULL,  
  `idDepartamento` INT NOT NULL,  
  `idEquip` INT NOT NULL,  
  PRIMARY KEY (`idDepartamento`, `idEquip`),  
  INDEX `idEquipamento_idx` (`idEquip` ASC) VISIBLE,  
  CONSTRAINT `idDepartamento`  
    FOREIGN KEY (`idDepartamento`)  
      REFERENCES `mydb`.`Departamento` (`idDepartamento`)  
      ON DELETE NO ACTION  
      ON UPDATE NO ACTION,  
  CONSTRAINT `idEquip`  
    FOREIGN KEY (`idEquip`)  
      REFERENCES `mydb`.`Equipamento` (`idEquipamento`)  
      ON DELETE NO ACTION  
      ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 55: Criação da tabela *Departamento_Equipamento*.

```
CREATE TABLE IF NOT EXISTS `mydb`.`Fornecedor_de_Peca` (  
  `id_fornecedor` INT NOT NULL,  
  `nome` VARCHAR(45) NOT NULL,  
  `contacto` INT(9) NOT NULL,  
  PRIMARY KEY (`id_fornecedor`))  
ENGINE = InnoDB;
```

Figura 56: Criação da tabela *Fornecedor_de_Peca*.



```
CREATE TABLE IF NOT EXISTS `mydb`.`Peca` (  
  `id_eca` INT NOT NULL,  
  `designacao` VARCHAR(45) NOT NULL,  
  `preco` DECIMAL(10,2) NOT NULL,  
  `id_intervencao` INT NOT NULL,  
  PRIMARY KEY (`id_eca`),  
  INDEX `id_intervencao_idx` (`id_intervencao` ASC) VISIBLE,  
  CONSTRAINT `id_intervencao`  
    FOREIGN KEY (`id_intervencao`)  
      REFERENCES `mydb`.`Intervencao_de_Manutencao` (`id_intervencao`)  
      ON DELETE NO ACTION  
      ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 57: Criação da tabela *Peca*.

```
CREATE TABLE IF NOT EXISTS `mydb`.`Intervencao_de_Manutencao` (  
  `id_intervencao` INT NOT NULL,  
  `data_inicio` DATETIME NOT NULL,  
  `data_fim` DATETIME NULL,  
  `descricao` VARCHAR(45) NOT NULL,  
  `custo_intervencao` DECIMAL(10,2) NOT NULL,  
  `custo_total` DECIMAL(10,2) NOT NULL,  
  `nr_ordem` INT NOT NULL,  
  `id_tecnico` INT NOT NULL,  
  PRIMARY KEY (`id_intervencao`),  
  INDEX `nr_ordem_idx` (`nr_ordem` ASC) VISIBLE,  
  INDEX `id_tecnico_idx` (`id_tecnico` ASC) VISIBLE,  
  CONSTRAINT `nr_ordem`  
    FOREIGN KEY (`nr_ordem`)  
      REFERENCES `mydb`.`Ordem_de_Servico` (`nr_ordem`)  
      ON DELETE NO ACTION  
      ON UPDATE NO ACTION,  
  CONSTRAINT `id_tecnico`  
    FOREIGN KEY (`id_tecnico`)  
      REFERENCES `mydb`.`Tecnico_de_manutencao` (`id_tecnico`)  
      ON DELETE NO ACTION  
      ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 58: Criação da tabela *Intervencao_de_Manutencao*.

```
CREATE TABLE IF NOT EXISTS `mydb`.`Tecnico_de_manutencao` (  
  `id_tecnico` INT NOT NULL,  
  `nome` VARCHAR(45) NOT NULL,  
  `data_nasc` DATE NOT NULL,  
  `contacto` INT(9) NOT NULL,  
  PRIMARY KEY (`id_tecnico`))  
ENGINE = InnoDB;
```

Figura 59: Criação da tabela *Tecnico_de_Manutencao*.



```
-----  
-- Table `mydb`.`Departamento`  
-----  
CREATE TABLE IF NOT EXISTS `mydb`.`Departamento` (  
  `idDepartamento` INT NOT NULL,  
  `localizacao` VARCHAR(45) NOT NULL,  
  `designacao` VARCHAR(45) NOT NULL,  
  PRIMARY KEY (`idDepartamento`))  
ENGINE = InnoDB;
```

Figura 60: Criação da tabela *Departamento*.

A.3 Povoamento

```
INSERT INTO Departamento_Equipamento (Data_Adjudicacao, idDepartamento, idEquip) VALUES  
( '2019-01-15', 1, 1),  
( '2022-02-20', 2, 2),  
( '2020-03-10', 3, 3),  
( '2022-04-05', 4, 4),  
( '2021-05-12', 5, 5);
```

Figura 61: Povoamento da tabela *Departamento_Equipamento*.

```
INSERT INTO Fornecedor_Peca (data_compra, id_eca, id_fornecedor) VALUES  
( '2024-01-12', 1, 1),  
( '2024-03-22', 3, 3),  
( '2024-04-16', 4, 4),  
( '2024-05-03', 5, 5),  
( '2022-02-01', 6, 5);
```

Figura 62: Povoamento da tabela *Fornecedor_Peca*.

```
INSERT INTO Fornecedor_de_Peca (id_fornecedor, nome, contacto) VALUES  
(1, 'Fornecedor A', 912345678),  
(2, 'Fornecedor B', 923456789),  
(3, 'Fornecedor C', 934567890),  
(4, 'Fornecedor D', 945678901),  
(5, 'Fornecedor E', 956789012);
```

Figura 63: Povoamento da tabela *Fornecedor_de_Peca*.

```
INSERT INTO Intervencao_de_Mantencao (id_intervencao, data_inicio, data_fim, descricao, custo_intervencao, custo_total, nr_ordem, id_tecnico) VALUES  
(1, '2024-01-15 08:00:00', NULL, 'Substituição de Tubo', 200.00, 300.00, 1, 1),  
(3, '2024-01-03 09:00:00', '2024-01-03 13:00:00', 'Calibração do Equipamento', 500.00, 700.00, 3, 3),  
(4, '2024-04-20 11:00:00', NULL, 'Substituição de Motor', 400.00, 550.00, 4, 2),  
(5, '2024-05-05 13:00:00', '2024-05-05 17:00:00', 'Reparo de Circuito', 300.00, 450.00, 5, 1),  
(6, '2022-02-15 09:00:00', '2022-02-15 10:00:00', 'Reparo de placa ', 200.00, 650.00, 6, 4),  
(2, '2023-01-25 15:00:00', '2022-01-26 09:00:00', 'Calibração do Equipamento', 400.00, 400.00, 7, 3);
```

Figura 64: Povoamento da tabela *Intervencao_de_Mantencao*.



```
INSERT INTO Ordem_de_Servico (nr_ordem, prioridade, descricao, estado_atual, data_criacao, idEquipamento) VALUES
(1, 1, 'Manutenção Preventiva', 'Em Andamento', '2024-01-10', 1),
(2, 2, 'Substituição de Peças', 'Pendente', '2024-05-29', 4),
(3, 3, 'Calibração', 'Concluída', '2024-03-20', 3),
(4, 1, 'Emergência', 'Em Andamento', '2024-04-15', 5),
(5, 2, 'Reparo', 'Concluída', '2024-05-01', 2),
(6, 1, 'Reparo', 'Concluída', '2022-02-10', 1),
(7, 3, 'Calibração', 'Concluída', '2023-01-20', 3);
```

Figura 65: Povoamento da tabela *Ordem_de_Servico*.

```
INSERT INTO Peca (id_peca, designacao, preco, id_intervencao) VALUES
(1, 'Tubo de Raio-X', 100.00, 1),
(3, 'Sensor de Calibração', 200.00, 3),
(4, 'Motor de Rotação', 150.00, 4),
(5, 'Placa de Circuito', 150.00, 5),
(6, 'Placa Central', 450.00, 6);
```

Figura 66: Povoamento da tabela *Peca*.

```
INSERT INTO Responsavel (idResponsavel, nome, data_nasc, id_Departamento) VALUES
(1, 'Carlos Silva', '1980-03-15', 1),
(2, 'Maria Santos', '1975-06-20', 2),
(3, 'Lara Mangualde', '1980-07-21', 3),
(4, 'Cristiano Ronaldo', '1970-09-22', 4),
(5, 'Alexandra Freitas', '1987-10-10', 5);
```

Figura 67: Povoamento da tabela *Responsavel*.

```
INSERT INTO Tecnico_de_manutencao (id_tecnico, nome, data_nasc, contacto) VALUES
(1, 'João Silva', '1990-05-15', 912345678),
(2, 'Ana Santos', '1985-08-20', 923456789),
(3, 'Pedro Costa', '1993-02-10', 934567890),
(4, 'Marta Oliveira', '1991-11-30', 945678901),
(5, 'Rui Sousa', '1988-07-25', 956789012);
```

Figura 68: Povoamento da tabela *Tecnico_de_Manutencao*.

A.4 Queries

Cálculo do Custo de todas as Intervenções num Intervalo de Tempo

```
-- 5 Custo de todas as intervencoes num intervalo de tempo
SELECT SUM(im.custo_total) AS custo_total_intervencoes
FROM Intervencao_de_Manutencao im
WHERE im.data_inicio BETWEEN '2023-01-01' AND '2024-12-31';
```

Figura 69: Query de Cálculo do Custo de todas as Intervenções num Intervalo de Tempo.



	custo_total_intervencoes
▶	2000.00

Figura 70: Resposta da *Query* de Cálculo do Custo de todas as Intervenção num Intervalo de Tempo.

Seleção das Intervenção cujo custo está acima da média de preços

```
-- 6 Intervencoes acima da media de preço
SELECT im.id_intervencao, im.descricao, im.custo_intervencao, im.custo_total, t.nome AS tecnico
FROM Intervencao_de_Mantencao im
JOIN Tecnico_de_mantencao t ON im.id_tecnico = t.id_tecnico
WHERE im.custo_total > (SELECT AVG(custo_total) FROM Intervencao_de_Mantencao);
```

Figura 71: *Query* de Seleção das Intervenção cujo custo está acima da média de preços.

	id_intervencao	descricao	custo_intervencao	custo_total	tecnico
▶	3	Calibração do Equipamento	500.00	700.00	Pedro Costa
	4	Substituição de Motor	400.00	550.00	Ana Santos

Figura 72: Resposta da *Query* de Seleção das Intervenção cujo custo está acima da média de preços.

Seleção dos Equipamentos comprados depois de 2022 (inclusive) com Número de Intervenção

```
-- 7 Equipamentos comprados depois de 2022 (inclusive) com numero de intervencoes
SELECT e.idEquipamento, e.designacao, e.data_aquisicao, COUNT(im.id_intervencao) AS num_intervencoes
FROM Equipamento e
LEFT JOIN Ordem_de_Servico os ON e.idEquipamento = os.idEquipamento
LEFT JOIN Intervencao_de_Mantencao im ON os.nr_ordem = im.nr_ordem
WHERE e.data_aquisicao > '2021-12-31'
GROUP BY e.idEquipamento, e.designacao, e.data_aquisicao;
```

Figura 73: *Query* de Seleção dos Equipamentos comprados depois de 2022 (inclusive) com Número de Intervenção.

	idEquipamento	designacao	data_aquisicao	num_intervencoes
▶	2	Impressora	2022-02-20	1
	4	Mesa de cirurgia	2022-04-05	0

Figura 74: Resposta da *Query* de Seleção dos Equipamentos comprados depois de 2022 (inclusive) com Número de Intervenção.



Seleção das Intervenções não realizadas pelo Pedro Costa

```
-- 8 Seleciona as intervencoes nao realizadas pelo pedro costa
SELECT im.id_intervencao, im.descricao,im.custo_intervencao, t.nome AS tecnico
FROM Intervencao_de_Manutencao im
JOIN Tecnico_de_manutencao t ON im.id_tecnico = t.id_tecnico
EXCEPT
SELECT im.id_intervencao, im.descricao, im.custo_intervencao,t.nome AS tecnico
FROM Intervencao_de_Manutencao im
JOIN Tecnico_de_manutencao t ON im.id_tecnico = t.id_tecnico
WHERE t.nome = 'Pedro Costa';
```

Figura 75: *Query* de Seleção das Intervenções não realizadas pelo Pedro Costa.

	id_intervencao	descricao	custo_intervencao	tecnico
▶	5	Reparo de Circuito	300.00	João Silva
	4	Substituição de Motor	400.00	Ana Santos
	1	Substituição de Tubo	200.00	João Silva

Figura 76: Resposta da *Query* de Seleção das Intervenções não realizadas pelo Pedro Costa.

Seleção das Intervenções não realizadas pelo Pedro Costa

```
-- 8 Seleciona as intervencoes nao realizadas pelo pedro costa
SELECT im.id_intervencao, im.descricao,im.custo_intervencao, t.nome AS tecnico
FROM Intervencao_de_Manutencao im
JOIN Tecnico_de_manutencao t ON im.id_tecnico = t.id_tecnico
EXCEPT
SELECT im.id_intervencao, im.descricao, im.custo_intervencao,t.nome AS tecnico
FROM Intervencao_de_Manutencao im
JOIN Tecnico_de_manutencao t ON im.id_tecnico = t.id_tecnico
WHERE t.nome = 'Pedro Costa';
```

Figura 77: *Query* de Seleção das Intervenções não realizadas pelo Pedro Costa.

	id_intervencao	descricao	custo_intervencao	tecnico
▶	5	Reparo de Circuito	300.00	João Silva
	4	Substituição de Motor	400.00	Ana Santos
	1	Substituição de Tubo	200.00	João Silva

Figura 78: Resposta da *Query* de Seleção das Intervenções não realizadas pelo Pedro Costa.



Número de Intervenções e Soma do Custo das Intervenções

```
-- 9 Numero de intervencoes e soma do custo das intervencoes  
SELECT t.nome AS tecnico, COUNT(im.id_intervencao) AS numero_intervencoes, SUM(im.custo_intervencao) AS total_custo_intervencao  
FROM Intervencao_de_Manutencao im  
JOIN Tecnico_de_manutencao t ON im.id_tecnico = t.id_tecnico  
GROUP BY t.nome;
```

Figura 79: *Query* do Número de Intervenções e Soma do Custo das Intervenções.

	tecnico	numero_intervencoes	total_custo_intervencao
▶	João Silva	2	500.00
	Pedro Costa	1	500.00
	Ana Santos	1	400.00

Figura 80: Resposta da *Query* do Número de Intervenções e Soma do Custo das Intervenções.